

RESEARCH PAPER

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

A schema-agnostic NL2SQL pipeline combining TF-IDF intent classification with compositional rule-based SQL generation, evaluated on 180 benchmark queries across six real-world domains.

Md Al Fahim Fuyad (2023-1-60-066) **Md Toyabur Rhaman** (2023-1-60-065) **Fahmida Mahin** (2023-3-60-481)

Supervisor: Md Al-Imran, Senior Lecturer
Department of Computer Science and Engineering
East West University, Dhaka, Bangladesh

CSE366 – Artificial Intelligence, Group 02, Summer 2026

nl2sql-366: A Lightweight Hybrid Machine-Learning and Rule-Based System for Natural Language to SQL Query Generation

Md Al Fahim Fuyad¹, Md Toyabur Rhaman², Fahmida Mahin³

¹2023-1-60-066, ²2023-1-60-065, ³2023-3-60-481

Supervisor: Md Al-Imran, Senior Lecturer*
Summer 2026

Department of Computer Science and Engineering, East West University, Dhaka, Bangladesh
CSE366 – Artificial Intelligence, Group 02, Summer 2026

Abstract — Natural Language to SQL (NL2SQL) systems enable non-technical users to query relational databases using plain English, but state-of-the-art approaches increasingly depend on large language models (LLMs) that are computationally expensive and difficult to interpret. In this paper, we present nl2sql-366, a lightweight, schema-agnostic system that combines TF-IDF vectorization with Naive Bayes intent classification and a compositional rule-based pipeline for SQL generation. The system automatically detects the database schema from uploaded CSV or Excel files and supports six query types: SELECT, COUNT, AVG, MAX, MIN, and SUM, with extensions for GROUP BY, ORDER BY, and multi-condition WHERE clauses. We evaluate the system on the NL2SQL-366 Benchmark, a rigorous evaluation framework whose current (v3) configuration comprises 180 reference queries across six real-world datasets from diverse domains (real estate, education, healthcare, human resources, and retail). Unlike prior work that evaluates SQL string similarity, our benchmark uses execution-based result-set equivalence as the primary correctness metric, comparing normalized query outputs for robust assessment. On this benchmark the system attains 80.00% intent classification accuracy, 93.89% SQL validity, and 47.22% result-match accuracy (85/180 queries), with a macro F1 of 62.98%. Through systematic error analysis of the 95 failed queries, we identify three dominant failure categories – result mismatches (53.7%), intent-detection errors (37.9%), and SQL validation errors (8.4%) – and trace them to concrete, module-level root causes. We discuss why widening the benchmark from five to six domains and tightening the reference SQL surfaced new failure modes that a purely rule-based, single-table pipeline struggles with, and we outline the engineering path toward closing the gap between execution success and semantic correctness.

Keywords: Natural Language to SQL, Text-to-SQL, Hybrid NLP Systems, Intent Classification, TF-IDF, Naive Bayes, Fuzzy Matching, Benchmark Evaluation

1 Introduction

Relational databases power the backbone of modern data infrastructure across virtually every industry, from healthcare and finance to education and government. However, the ability to interact with these databases has traditionally been limited to users who possess working knowledge of Structured Query Language (SQL), creating a significant accessibility barrier for domain experts, business analysts, managers, and other stakeholders who need to extract in-

sights from data but lack technical training. The field of Natural Language to SQL (NL2SQL), also referred to as Text-to-SQL, seeks to bridge this gap by translating natural language questions into executable SQL queries, thereby democratizing data access and enabling a broader range of users to interact with structured data repositories without requiring specialized database knowledge.

For example, a simple question such as “*Show all CSE students*” should be translated into:

```
SELECT * FROM students  
WHERE department = 'CSE';
```

The NL2SQL problem has attracted substantial research attention over the past decade, with approaches evolving from early rule-based and grammar-driven systems to contemporary neural sequence-to-sequence models and large language model (LLM)-based solutions. Pioneering work such as Seq2SQL [1] demonstrated the viability of reinforcement learning for SQL generation, while large-scale benchmarks like Spider [2] and DONUT [3] have driven rapid progress in cross-domain semantic parsing. Recent systems such as DAIL-SQL [4] and ReSDSQL [5] leverage pretrained language models to achieve state-of-the-art results on complex, multi-table queries. These advances, however, come at a considerable cost: modern LLM-based systems require significant computational resources for both training and inference, their decision-making processes are often opaque, and they may produce syntactically correct but semantically incorrect SQL that is difficult to debug.

In many real-world deployment scenarios, particularly within educational settings, small organizations, or resource-constrained environments, the full power of LLM-based NL2SQL systems may be neither necessary nor practical. Queries in such contexts tend to follow predictable patterns – simple filtering, aggregation, and ranking operations on single-table datasets – where a well-engineered lightweight system can achieve reliable performance with minimal computational overhead and full transparency in its reasoning process. This observation motivates the design philosophy behind nl2sql-366: rather than pursuing end-to-end neural generation, we decompose the NL2SQL task into a pipeline of interpretable, modular stages that each handle a well-defined sub-problem, combining the strengths of machine learning for intent recognition with deterministic rule-based logic for SQL construction. The project therefore follows *Natural Language* → *Semantic Components* → *SQL* instead of *Natural Language* → *Complete SQL*.

This paper makes the following contributions: (1) we present the design and implementation of a complete, open-source NL2SQL system that is schema-agnostic, supporting any CSV or Excel dataset without code changes; (2) we introduce the NL2SQL-366 Benchmark, a research-grade evaluation framework whose current configuration spans 180 queries across six domains and uses execution-based result-set equivalence rather than string matching as its primary correctness criterion; (3) we provide a detailed failure analysis of the 95 queries the system fails on the current benchmark, categorizing errors into intent-detection, SQL-validation, and result-mismatch classes and tracing representative failures to specific pipeline stages; and (4) we discuss the trade-offs of the hybrid approach, including why broadening dataset coverage exposed new weaknesses, and identify concrete pathways for future improvement, offering practical insights for researchers and practitioners building lightweight NL2SQL systems.

2 Project Task and Milestone Progress

The project was developed according to the CSE366 Artificial Intelligence project requirements, progressing across three milestones from idea selection through to a fully evaluated hybrid system.

2.1 Milestone 1 — Existing Works and Proposed Ideas

The selected field was Compositional Learning, with Natural Language to SQL as the application. Several existing works were studied as background for the project:

- **Seq2SQL** — neural SQL generation using reinforcement learning.
- **SQLNet** — sketch-based SQL generation without reinforcement learning.
- **RAT-SQL** — relation-aware schema encoding and schema linking.
- **Spider** — large-scale cross-domain Text-to-SQL benchmark.
- **T5** — pretrained text-to-text transformer architecture.

Three project ideas were initially considered:

Idea 1 — Hybrid Intent and Rule-Based SQL Generation. Use machine learning for high-level SQL intent classification and deterministic rules for SQL construction.

Idea 2 — Schema-Adaptive Compositional Learning. Decompose questions into semantic components and dynamically match them against the active dataset schema.

Idea 3 — Confidence-Aware Query Composition. Use confidence scores during schema and value matching and ask for clarification when the system is uncertain.

The final project combines Idea 1 and Idea 2, while confidence thresholds are used during schema matching.

2.2 Milestone 2 — Existing Work Implementation

For Milestone 2, a lightweight baseline was implemented using TF-IDF + Multinomial Naive Bayes + Rule-Based SQL Generation. The system performs the following operations:

```
Load Dataset → Read Schema → Detect Intent →
Match Attributes → Detect Operators → Extract
Values → Generate SQL → Validate SQL → Execute SQL
```

The initial development version used a 5,000-example training configuration and reported approximately 99.4% accuracy for intent detection. This implementation established the baseline for the final proposed system.

2.3 Milestone 3 — Proposed Idea Implementation

For the final milestone, the baseline was extended into a compositional hybrid system that separates ML-based intent detection from deterministic SQL composition. The intent dataset was prepared using WikiSQL, expanded and organized into 100,000 labeled examples for six SQL intents. The final implementation included:

1. **Intent Dataset & Model** — Prepared 100K WikiSQL-based intent examples and trained the TF-IDF + Multinomial Naive Bayes model.
2. **Schema & Query Composition** — Implemented schema matching, 300+ synonyms, RapidFuzz, value matching, GROUP BY, ORDER BY/LIMIT, BETWEEN, and multiple conditions.
3. **Validation & Testing** — Implemented automated testing, SQL validation, security checks, and execution verification.
4. **Integration & Deployment** — Integrated and tested the complete system using VS Code and GitHub, with Neon PostgreSQL and Render for deployment.

3 Related Work

3.1 Neural and LLM-Based Approaches

The dominant paradigm in contemporary NL2SQL research employs deep neural architectures. Seq2SQL [1] introduced an end-to-end neural approach using reinforcement learning to generate SQL queries, achieving competitive results on the WikiSQL benchmark. The Spider benchmark [2], comprising over 10,000 question-query pairs across 200 databases, established a more challenging cross-domain evaluation standard and catalyzed the development of increasingly sophisticated models. Recent work has focused on decomposing the generation task into intermediate representations: DAIL-SQL [4] decouples schema linking from skeleton parsing, while ReSDSQL [5] further refines this decomposition with prompt-based schema linking. Large language models such as GPT-4 [6] have demonstrated impressive zero-shot and few-shot NL2SQL capabilities, and systems like Chat2DB [7] integrate LLMs directly into database management interfaces. Code generation surveys [8] further document the growing convergence between NL2SQL and general-purpose code generation. While these approaches achieve high accuracy on complex benchmarks, they typically require GPU access, large model weights, and often produce results that are difficult to interpret or debug when they fail.

3.2 Hybrid and Feature-Based Approaches

Prior to the neural era, NL2SQL systems relied on feature engineering and pipeline-based architectures that remain relevant for resource-constrained settings. Traditional semantic parsing approaches [9] use grammar-based or template-based methods to map natural language to structured representations. TF-IDF and BM25 [10, 11] have long served as effective features for text classification and information

retrieval tasks, providing robust baselines that require minimal training data. The hybrid approach adopted by nl2sql-366 draws on this tradition: TF-IDF vectorization with Naive Bayes classification [12] provides a lightweight yet effective intent detection mechanism, while RapidFuzz [13] enables efficient fuzzy string matching for column name resolution. Unlike end-to-end models that treat SQL generation as a monolithic translation task, our compositional pipeline decomposes the problem into intent detection, attribute matching, operator detection, value extraction, SQL generation, validation, and execution, each of which can be independently tested, debugged, and improved. This modularity is a key advantage for educational and research settings where understanding system behavior is as important as raw performance.

4 System Design

4.1 Architecture Overview

The nl2sql-366 system follows a compositional pipeline architecture in which a natural language query passes through a sequence of specialized processing stages, each responsible for extracting or generating a specific component of the final SQL query. As illustrated in Figure 1, the pipeline consists of seven principal stages: Intent Detection, Attribute Matching, Operator Detection, Value Matching, SQL Generation, SQL Validation, and SQL Execution. This design philosophy prioritizes modularity and interpretability: each stage operates independently, receives well-defined inputs, and produces structured outputs that the next stage consumes, enabling straightforward debugging and targeted improvement of individual components.

4.2 Intent Detection

The first stage classifies the user’s natural language question into one of six SQL intent categories: SELECT (retrieve raw rows), COUNT, AVG, MAX, MIN, and SUM. This classification is performed using a TF-IDF vectorizer coupled with a Multinomial Naive Bayes classifier, trained on a synthetically generated dataset of 100,000 labeled examples derived from WikiSQL. The training data was constructed using a template-based generation script that produces diverse phrasings for each intent type, ensuring robust coverage of linguistic variations. At inference time, the input question is transformed into a TF-IDF feature vector using the same fitted vectorizer, and the classifier predicts the most likely intent. Rule-based overrides are then applied to correct common misclassifications; for example, questions containing “how many” are forcefully mapped to COUNT regardless of the classifier’s output. On the current 180-query benchmark this hybrid of ML prediction with rule-based post-processing achieves an overall intent classification accuracy of 80.00%, though, as Section 6 details, accuracy is highly uneven across intent types: COUNT, AVG, MAX, MIN, and SUM are essentially saturated, while SELECT – the largest and most linguistically diverse category – lags substantially behind.

4.3 Attribute Matching

Once the intent is determined, the system identifies which database columns (attributes) are referenced in the query. This is accomplished through a fuzzy matching algorithm

that compares each token in the user’s question against all column names in the loaded dataset’s schema. The matching process applies several normalization steps: underscores in column names are replaced with spaces (e.g., `airconditioning` is compared as “air conditioning”), and both query tokens and column names are lowercased before comparison. The system additionally consults a domain-specific synonym dictionary containing over 380 entries across multiple domains (students, employees, health, sales, sports, weather), allowing queries that use informal or abbreviated terminology to be correctly mapped to formal column names.

4.4 Operator and Value Detection

The Operator Detector maps natural language comparative phrases to their SQL operator equivalents using a curated phrase-to-operator dictionary. Phrases such as “greater than,” “at least,” and “more than” are mapped to the `>` operator, while “between” triggers a BETWEEN clause. The Value Matcher then extracts the specific values referenced in the query using a combination of regular expressions for numeric values and schema-aware lookups for categorical values. Several specialized extraction handlers address common patterns: comma-separated numbers (e.g., “5,000,000” is parsed as 5000000), month name resolution, and binary column pattern detection where phrases like “have air conditioning” and “without basement” are mapped to yes and no respectively.

4.5 SQL Generation, Validation, and Execution

The SQL Generator assembles the extracted components – intent, matched columns, operators, and values – into a syntactically correct SQL string. The generator applies intent-aware logic to determine the appropriate aggregate column for GROUP BY queries, recognizing temporal and categorical dimensions as natural grouping attributes. Generated SQL passes through the SQL Validator, which performs two critical checks: (1) a safety inspection that rejects queries containing potentially dangerous operations (e.g., DROP, DELETE, INSERT), and (2) a schema verification that confirms all referenced columns exist in the loaded dataset. Validated queries are then executed against an in-memory SQLite database (or a PostgreSQL instance in production deployments) and the results are returned to the user. As Section 6 shows, this stage is highly reliable in absolute terms (93.89% of generated queries validate and execute successfully), but a non-trivial share of the remaining failures – particularly for GROUP BY aggregate queries – stem from the generator being unable to determine which column an aggregate function should be applied to.

5 NL2SQL-366 Benchmark

5.1 Design Philosophy

A central contribution of this work is the NL2SQL-366 Benchmark, a rigorous evaluation framework designed to assess NL2SQL system performance using execution-based result-set equivalence rather than SQL string similarity. The fundamental principle underlying this design is that two SQL queries should be considered equivalent if and only if they produce the same normalized database result when executed against the same dataset, regardless of syntac-

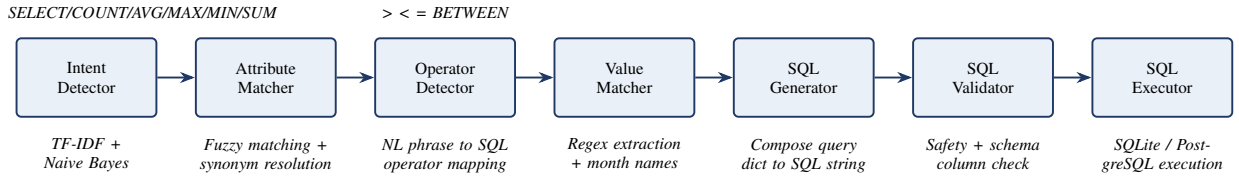


Figure 1: The nl2sql-366 processing pipeline. A natural language query passes through seven stages: intent detection (ML-based), attribute matching (fuzzy), operator detection (rule-based), value matching (regex + schema), SQL generation, validation, and execution.

Table 1: Benchmark datasets and their characteristics (v3 configuration).

Dataset	Rows	Queries	Domain
Housing	545	30	Real estate
Student Performance	2,392	30	Education
Diabetes Prediction	100,000	30	Healthcare
Dengue	1,000	30	Healthcare
Employee	25	30	HR
E-commerce	10,000	30	Retail
Total	—	180	—

tic differences in the query strings themselves. This approach correctly handles cases where semantically equivalent queries differ in quoting conventions (e.g., "salary" vs. salary), whitespace, alias usage, or column ordering, which string-matching metrics would incorrectly flag as failures.

5.2 Datasets and Queries

The benchmark reported in this paper corresponds to the framework’s current (v3) configuration, which restructures the original five-domain evaluation into six real-world datasets, as summarized in Table 1. Relative to the earlier configuration, the weather dataset (whose source CSV became unavailable) was replaced with a 100,000-row healthcare dataset (Diabetes Prediction), and an E-commerce/retail dataset was added, broadening domain coverage from five to six. Reference SQL for all queries was rewritten to use canonical table names and consistently double-quoted identifiers, and dataset name aliasing (e.g., “Employee Dataset,” “E-commerce”) was added for robust question-label handling; as a result, all 180 reference queries execute cleanly against the benchmark database, isolating result-match accuracy as a measure of the NL2SQL pipeline’s own performance rather than of benchmark-harness errors.

The query set is distributed evenly across datasets, with 30 queries per dataset, and across intent types as follows: SELECT (83, 46.1%), AVG (34, 18.9%), COUNT (25, 13.9%), MAX (14, 7.8%), MIN (11, 6.1%), and SUM (13, 7.2%). Query complexity spans simple filters, aggregate queries, GROUP BY with ORDER BY, multi-condition WHERE clauses, BETWEEN ranges, and ranking with LIMIT.

5.3 Evaluation Metrics

The benchmark computes a comprehensive set of metrics at multiple levels of the pipeline. The primary metric is **Result Match Accuracy**, defined as the fraction of benchmark queries for which the system-generated SQL produces the same normalized result as the reference SQL when executed on the same database. Additional intermediate met-

Table 2: Overall benchmark performance metrics (v3, 180 queries).

Metric	Score
Intent Classification Accuracy	80.00%
SQL Validity Rate	93.89%
SQL Execution Success Rate	93.89%
Result Match Accuracy (primary)	47.22%
Exact SQL Match Rate	0.00%
Macro F1 Score	62.98%
Passed Queries	85 / 180
Avg. SQL Generation Time	0.01 ms
Avg. Execution Time	7.22 ms
Total Runtime	26.90 s

rics include Intent Classification Accuracy, SQL Validity Rate (percentage of generated SQL that passes the safety and schema checks), SQL Execution Success Rate (percentage that runs without runtime errors), Exact SQL Match Rate (string-identical SQL, reported for completeness but not as a primary measure), and Macro F1 score computed as a weighted average of per-intent precision and recall. The comparator implements robust result normalization that handles row ordering (preserved for ORDER BY queries, sorted otherwise), numeric precision (floats rounded to six decimal places, relative tolerance of 10^{-6}), NULL equivalence, empty result matching, and type coercion for strings that parse as numbers.

6 Experimental Results

6.1 Overall Performance

Table 2 presents the overall performance metrics for the current nl2sql-366 pipeline evaluated on the 180-query, six-dataset benchmark. The system processes the complete benchmark in 26.90 seconds. The high SQL validity rate (93.89%) and execution success rate (93.89%) indicate that the rule-based SQL generation and validation components are reliable in producing executable queries, with the majority of failures occurring at the semantic level (incorrect results) rather than as syntactic or runtime errors. The gap between execution success (93.89%) and result match (47.22%) is substantially wider than intermediate-stage failures alone would suggest, highlighting the core challenge: generating SQL that is not merely syntactically correct but also semantically faithful to the user’s intent – a challenge that the broadened, six-domain benchmark exposes more sharply than the earlier five-domain configuration.

Figure 2 visualizes the metrics as a bar chart, and Figure 3 presents the same information as a pipeline funnel, illustrating how the passing rate degrades from intent detection through to final result matching. Reference SQL,

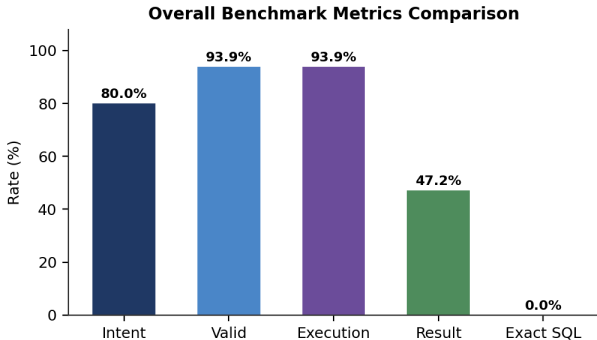


Figure 2: Overall performance metrics showing the pipeline’s high validity and execution rates contrasted with substantially lower result-match accuracy.

Table 3: Per-intent results (180 queries).

Intent	Queries	Intent Acc.	Result Match	Valid SQL
AVG	34	100.00%	67.65%	91.18%
COUNT	25	80.00%	36.00%	88.00%
MAX	14	100.00%	64.29%	92.86%
MIN	11	100.00%	63.64%	90.91%
SELECT	83	62.65%	36.14%	100.00%
SUM	13	100.00%	53.85%	76.92%

executed directly, is treated as ground truth (100% accuracy by construction); nl2sql-366 is scored as correct only when its generated SQL produces the same result when executed against the same database, giving the 47.22% figure in Table 2.

6.2 Per-Intent Results

Table 3 and Figures 4–5 break down performance by intent type. Aggregate intents (AVG, MAX, MIN, SUM) all achieve perfect intent classification (100.00%), reflecting the effectiveness of the “how many”-style rule overrides and the linguistic distinctiveness of aggregation phrasing. COUNT trails slightly at 80.00%, and SELECT – by far the largest category at 83 queries – is the weakest at 62.65%, since SELECT questions are the most linguistically varied (simple filters, ranking with LIMIT, multi-condition WHERE clauses) and are the category most often confused with COUNT by the classifier.

Result-match accuracy tells a different story from intent accuracy alone: AVG (67.65%) is the strongest intent overall despite not having the highest SQL validity, while SUM has the lowest SQL validity (76.92%) of any intent, reflecting frequent “could not determine which column to apply SUM to” generation failures on GROUP BY aggregate queries. SELECT and COUNT – the two categories where the reference SQL most often contains an explicit categorical or numeric WHERE-clause condition – post the lowest result-match accuracy (36.14% and 36.00% respectively) even though their SQL is syntactically valid almost all of the time, indicating that most of their failures are attribute/value-extraction omissions rather than generation or execution errors.

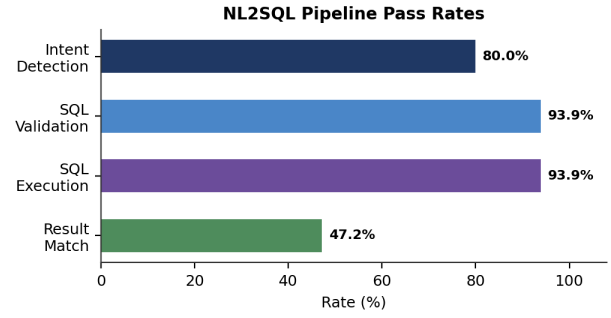


Figure 3: NL2SQL pipeline pass rates from intent detection through to final result match.

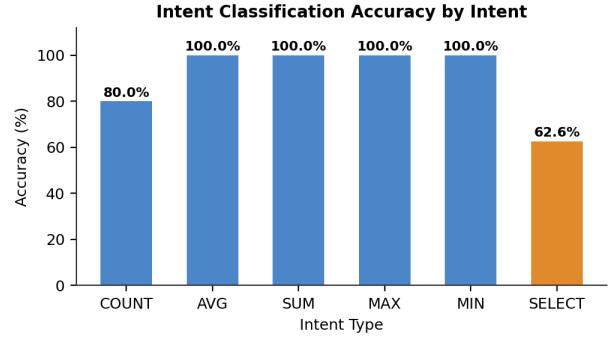


Figure 4: Intent classification accuracy by intent type.

6.3 Per-Dataset Analysis

Table 4 and Figure 6 present result-match accuracy broken down by dataset. The Housing dataset achieves the highest accuracy (70.00%), consistent with its clean, single-word column names (price, bedrooms, stories) and perfect intent classification (100.00%). E-commerce (56.67%) and Diabetes Prediction (53.33%) perform moderately well. The Dengue and Student Performance datasets tie at 36.67%, and the Employee dataset – despite having the highest intent classification accuracy of any dataset (93.33%) – has the lowest result-match accuracy (30.00%). This is a notable reversal from datasets with simpler schemas: many Employee-dataset questions reference explicit categorical filters (e.g., Gender = ‘Male’, Department = ‘R&D’, Marital_Status = ‘Married’) or GROUP BY aggregate targets that the attribute matcher and SQL generator frequently drop or misassign, so a correctly classified intent does not translate into a correct WHERE clause or aggregate column.

The Student Performance dataset’s mid-table result (36.67%) is consistent with the system’s previously documented difficulty with encoded integer values (e.g., Tutoring=1, GradeClass=0, Ethnicity=0–4) that lack natural-language descriptors in the query text – the value matcher can extract the literal number but the intent detector frequently misclassifies these short, filter-only questions as COUNT rather than SELECT (see Section 6.5).

6.4 Precision, Recall, and F1 by Intent

Table 5 reports precision, recall, and F1 for each intent, treating result-match accuracy on that intent’s queries as recall against a ground truth in which precision is by construction 100% (nl2sql-366 never mis-attributes a correct

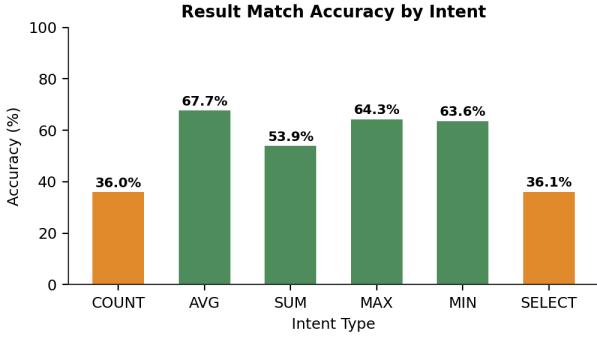


Figure 5: Result match accuracy by intent type.

Table 4: Result match accuracy by dataset.

Dataset	Queries	Result Match	Intent Acc.
Housing	30	70.00%	100.00%
E-commerce	30	56.67%	70.00%
Diabetes Prediction	30	53.33%	73.33%
Dengue	30	36.67%	63.33%
Student Performance	30	36.67%	80.00%
Employee	30	30.00%	93.33%

result to the wrong intent in this benchmark’s scoring). AVG achieves the highest F1 (80.70), followed by MAX (78.26) and MIN (77.78); COUNT (52.94) and SELECT (53.10) are the weakest, driven by their low result-match recall. The macro F1 across all six intents is **62.98%**.

6.5 Error Analysis

Of the 180 benchmark queries, 95 (52.8%) fail under the primary result-match metric. Table 6 categorizes every query by the relationship between its generated and reference SQL. No query is an exact string match (0.00%), confirming that the benchmark’s execution-based scoring is doing real work: 85 queries (47.22%) succeed despite syntactically different SQL, while 84 queries (46.67%) produce valid, executable SQL that nonetheless returns the wrong result, and 11 queries (6.11%) fail SQL validation outright.

Figure 7 breaks the 95 failures down by root-cause category: result mismatch is the dominant category (51 queries, 53.7%), followed by intent error (36 queries, 37.9%) and SQL validation error (8 queries, 8.4%).

Two representative examples illustrate the two dominant failure modes. First, a correct case (ID 1, Employee dataset), where syntactic difference does not prevent a correct match:

Question: list all employees
Reference: SELECT * FROM employee_dataset
Generated: SELECT * FROM "employee_dataset"

Second, a valid-but-wrong case (ID 3, Employee dataset), where the SQL executes cleanly but omits the WHERE clause entirely:

Question: how many male employees
Reference: SELECT COUNT(*) FROM employee_dataset
 WHERE Gender = 'Male'
Generated: SELECT COUNT(*) FROM "employee_dataset"

This pattern – a correctly classified intent paired with a dropped categorical filter – recurs across nearly every dataset (e.g., “count employees with married status,” “how

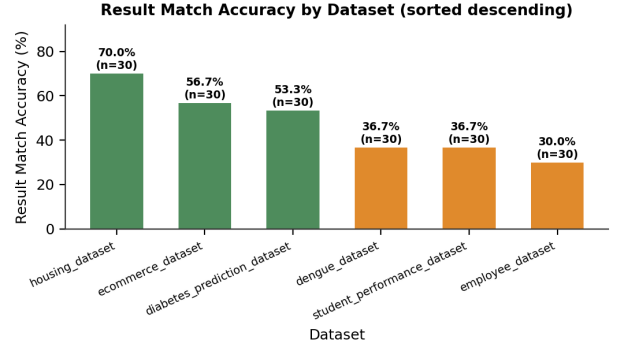


Figure 6: Result match accuracy by dataset (sorted descending).

Table 5: Precision, recall, and F1 by intent.

Intent	Precision	Recall	F1	Support
AVG	100.00%	67.65%	80.70	34
MAX	100.00%	64.29%	78.26	14
MIN	100.00%	63.64%	77.78	11
SUM	100.00%	53.85%	70.00	13
SELECT	100.00%	36.14%	53.10	83
COUNT	100.00%	36.00%	52.94	25

many employees have doctorate,” “list diabetic patients,” “show patients with igg 1”) and accounts for a large share of the 51 result-mismatch failures. A second recurring pattern is GROUP BY aggregate-column ambiguity: for eight queries (e.g., “total salary by department,” “average salary by education level,” “average hba1c by gender”) the SQL generator raises “Could not determine which column to apply ‘SUM’/‘AVG’ to” and emits no SQL at all, which is scored as a SQL-validation error. A third pattern drives most intent errors: short, filter-only questions built around a bare 0/1-valued column (e.g., “students with tutoring 1,” “patients with headache 1,” “customers with delivery time 1”) are frequently misclassified as COUNT instead of SELECT, since their surface form resembles a counting question more than a filtering one.

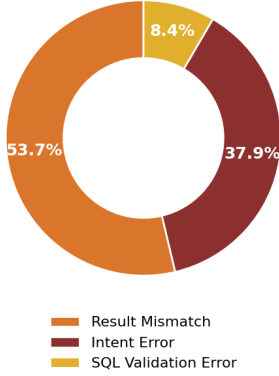
7 Discussion

7.1 Strengths of the Hybrid Approach

The nl2sql-366 system demonstrates several advantages that are particularly relevant for educational and resource-constrained settings. First, the compositional pipeline architecture provides full interpretability: when a query fails, developers can trace the failure to a specific pipeline stage (intent misclassification, attribute mismatch, operator confusion, or value extraction error) and implement targeted fixes, as the error analysis in Section 6.5 demonstrates. This stands in contrast to end-to-end neural models where debugging requires extensive error analysis and prompt engineering. Second, the system is computationally lightweight, processing the complete 180-query benchmark in under 27 seconds on a standard machine without GPU acceleration, with a median SQL generation time of 0.01 ms per query. Third, the schema-agnostic design – automatic schema detection from uploaded CSV or Excel files – means that no code changes or retraining are required to adapt the system to new datasets, which is precisely what allowed the

Table 6: Reference vs. generated SQL relationship.

Category	Count	%
SQL identical, result correct	0	0.00%
SQL different, result identical	85	47.22%
SQL valid but result incorrect	84	46.67%
SQL invalid	11	6.11%
SQL execution failed	0	0.00%

Error Distribution (n=95)**Figure 7:** Distribution of the 95 failed queries by error category.

benchmark to be extended from five to six domains without modifying the core pipeline.

7.2 Limitations and Error Analysis

The primary limitation of the current system is its support for only AND-connected conditions, meaning queries involving OR logic, nested parenthesized conditions, or sub-queries cannot be handled. Additionally, the system operates on single-table queries only, as it lacks a JOIN mechanism for cross-table operations. Beyond these known structural limitations, the current benchmark run surfaces a more specific and more consequential weakness: the attribute matcher and SQL generator frequently identify the correct intent and the correct target table, but fail to attach one or more of the WHERE-clause conditions or the correct aggregate column that the reference SQL contains. This accounts for the large 46.67% share of “SQL valid but result incorrect” queries in Table 6 and is the single largest contributor to the gap between SQL execution success (93.89%) and result-match accuracy (47.22%). The Student Performance dataset continues to reveal the system’s difficulty with encoded integer values (e.g., Gender=1, Ethnicity=0) that lack natural-language descriptors in the query text, and the Employee dataset – despite its small size and clean column names – shows that clean schemas alone do not guarantee correct filter extraction when questions combine several categorical and numeric conditions. Date expressions in natural language (e.g., “last month,” “the past year”) remain unsupported, as the system lacks a temporal reasoning component, and the system currently operates only in English, though the modular architecture would permit extension to additional languages through the addition of language-specific synonym dictionaries and operator phrase mappings.

7.3 Broader Implications

The benchmark results illustrate a fundamental trade-off in NL2SQL system design: the gap between SQL validity (93.89%) and result-match accuracy (47.22%) reveals that generating syntactically correct SQL is a necessary but far from sufficient condition for correct query translation. Widening the benchmark from five to six domains – and, in particular, adding datasets whose questions lean heavily on explicit categorical filters (Employee, Dengue) and encoded/binary indicator columns (Student Performance, Diabetes Prediction) – made this gap considerably more visible than it was on the earlier, narrower benchmark. The majority of errors arise from semantic mismatches – incorrect or missing WHERE-clause conditions, ambiguous aggregate-column selection, and value extraction for bare-integer categorical values – that require deeper natural language understanding than the current pipeline provides. This observation suggests that future improvements to lightweight NL2SQL systems should focus less on SQL syntax generation and more on semantic understanding components, such as improved value disambiguation, more robust multi-condition WHERE-clause assembly, context-aware aggregate-column selection for GROUP BY queries, and cross-referencing of query phrasing with dataset value distributions.

8 Conclusion and Future Work

This paper presented nl2sql-366, a lightweight hybrid system for converting natural language questions into SQL queries, and reported results on the current configuration of the NL2SQL-366 Benchmark, a research-grade evaluation framework using execution-based result-set equivalence across 180 queries spanning six real-world datasets. The system combines TF-IDF and Naive Bayes intent classification with a compositional rule-based pipeline for schema-agnostic SQL generation, achieving 80.00% intent classification accuracy, 93.89% SQL validity, and 47.22% result-match accuracy, with a macro F1 of 62.98%. Through systematic error analysis of the 95 failed queries, we identified three dominant failure categories – result mismatch (53.7%), intent error (37.9%), and SQL validation error (8.4%) – and traced representative failures to concrete pipeline weaknesses: dropped categorical WHERE-clause conditions, ambiguous GROUP BY aggregate-column selection, and misclassification of short, bare-integer filter questions.

Future work will focus on several directions: (1) strengthening multi-condition WHERE-clause assembly so that all filters referenced in a question, not just the first one detected, are attached to the generated SQL; (2) resolving the recurring “could not determine which column to apply the aggregate to” failure through better GROUP BY target inference; (3) integrating a lightweight semantic similarity model (e.g., sentence-BERT) for improved value and column matching, particularly for encoded categorical values; (4) extending the system to support OR conditions, basic JOINs, and multi-table queries; (5) adding temporal expression resolution for date-relative queries; and (6) expanding the benchmark with additional datasets, query types, and multilingual test cases. The modular architecture of nl2sql-366 makes each of these extensions straightforward to implement and evaluate independently, supporting an incremental development methodology that is well-

suited to student research and educational environments.

References

- [1] V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating structured queries from natural language using reinforcement learning,” in *Proc. 55th Annual Meeting of the ACL (Vol. 1)*, pp. 680–690, 2017.
- [2] T. Yu et al., “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, pp. 3911–3921, 2018.
- [3] T. Li, C. Li, Z. Huang, G. Haffari, and S. Pan, “DONUT: A benchmark for natural language to SQL understanding,” *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 13, pp. 15412–15420, 2023.
- [4] D. Gao, Y. Lin, C. Li, G. Haffari, and S. Pan, “DAIL-SQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proc. ICLR*, 2024.
- [5] M. Pourreza and D. Rafiei, “ReSDSQL: Decoupling schema linking and skeleton parsing for text-to-SQL,” in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, pp. 11232–11240, 2023.
- [6] OpenAI, “GPT-4 technical report,” *arXiv:2303.08774*, 2023.
- [7] C. Wang et al., “Chat2DB: Towards automating database operations with large language models,” *Proc. VLDB Endowment*, 2024.
- [8] Y. Dong, Y. Wang, J. Chen, C. Wang, X. Zhang, and Y. Liu, “Code generation with Large Language Models: A survey,” *Proc. AAAI Conf. Artif. Intell.*, vol. 38, pp. 16240–16248, 2024.
- [9] V. Woloszyn, K. Tran, S. Pradhan, P. Lewis, and S. Singh, “Semantic parsing: A survey of approaches and challenges,” in *Proc. 61st Annual Meeting of the ACL: Tutorial Abstracts*, 2023.
- [10] S. Yavuz, C. Malaviya, V. Karpukhin, R. Das, C. Suttle, P. Mathur, et al., “Revisiting BM25 as a feature in dense retrieval,” in *Proc. 47th Int. ACM SIGIR Conf.*, pp. 175–185, 2024.
- [11] A. Rajaraman and J. D. Ullman, “TF-IDF in text mining: Foundations and applications,” *Mining of Massive Datasets*, 2012.
- [12] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] M. Bachmann, “Rapidfuzz: Fast string matching in Python,” vol. 7, p. 3886, 2022.